CPLP - Práctico 3

Ejercicio 1: ¿Qué define la semántica?

1) La semántica describe el significado de los símbolos, palabras y frases de un lenguaje, ya sea un lenguaje natural o informático.

Ejercicio 2:

a. ¿Qué significa compilar un programa?
b. Describa brevemente cada uno de los pasos necesarios para compilar un programa.
c. ¿En qué paso interviene la semántica y cual es su importancia dentro de la compilación?

2) a) Compilar un programa significa traducir el código fuente escrito en un lenguaje de alto nivel a, generalmente, lenguaje de máquina. Este proceso es realizado por un compilador, que analiza todo el código fuente antes de ejecutarlo y genera un ejecutable o código intermedio.

Una vez compilado, el programa se puede ejecutar. Esto lo diferencia de los lenguajes interpretados, donde el código fuente se traduce línea por línea en tiempo de ejecución.

b)

1- Etapa de Análisis (Vinculadas al código fuente):

- Análisis léxico (Programa Scanner): Es un proceso que lleva tiempo, se encarga de hacer el análisis a nivel de palabra (LEXEMA) y divide el programa en sus elementos/categorías: identificadores, delimitadores, símbolos especiales, operadores, números, palabras clave, palabras reservadas, comentarios, etc. Además, analiza el tipo de cada uno para ver si son TOKENS válidos y filtra comentarios y separadores (como: espacios en blanco, tabulaciones, etc.)
  Lleva una tabla para la especificación del analizador léxico que incluye cada categoría, el conjunto de atributos y acciones asociadas, además, pone los identificadores en la tabla de símbolos y reemplaza cada símbolo por su entrada en la tabla.
  Genera errores si la entrada no coincide con ninguna categoría léxica y el resultado de este paso será el descubrimiento de los ítems léxicos o tokens y la detección de errores.
- Análisis sintáctico (Programa Parser): El análisis se realiza a nivel de sentencia/estructuras. Tiene como objetivo identificar las estructuras de las sentencias, declaraciones, expresiones, etc. presentes en su entrada usando los tokens del analizador léxico.
  Estas estructuras se pueden representar mediante el árbol de análisis sintáctico o árbol de derivación, que explica cómo se puede derivar la cadena de entrada en la gramática que especifica el lenguaje, validando qué pertenece o no a la gramática en pos de ver que lo que entra es correcto.
  El analizador sintáctico (Programa Parser) se alterna/interactúa con el análisis léxico y análisis semántico. Y usualmente usan técnicas de gramáticas formales.
- Análisis semántico (Programa de Semántica estática): Para realizar este análisis primero se debe pasar correctamente el Scanner y el Parser.
  Esta etapa se considera como una fase medular, es decir, una de las más importantes, se encarga de procesar las estructuras sintácticas, realizar comprobación de tipos (aplica gramática de atributos), agregar a la tabla de símbolos los descriptores de tipos, realizar comprobaciones de duplicados, problemas de tipos, comprobaciones de nombres, etc.
  Además, agrega información implícita y la estructura del código ejecutable continúa tomando forma.
  Esta etapa es el nexo entre etapas inicial y final del compilador (Análisis y Síntesis).

en un código compilado no comentarios ni rellenos innecesarios

Entre 1- y 2- ocurre *Generación de código intermedio*: Es realizar la transformación del "código fuente" en una representación de "código intermedio" para una máquina abstracta. Queda una representación independiente de la máquina en la que se va a ejecutar el programa. El código intermedio más habitual es el código de 3-direcciones.

2- Etapa de Síntesis (Vinculadas a características del código objeto, del hardware y la arquitectura):

- Optimización del código: No se hace siempre y no lo hacen todos los compiladores. Estos programas pueden ser herramientas independientes, o estar incluidas en los compiladores e invocarse por medio de opciones de compilación.
  Existen diversas formas y cosas que se pueden optimizar como por ejemplo elegir entre velocidad de ejecución y tamaño del código ejecutable, generar código para un microprocesador específico dentro de una familia de procesadores, eliminar la comprobación de rangos o desbordamientos de pila, evaluación para expresiones booleanas, eliminación de código muerto o no utilizado, eliminación de funciones no utilizadas, etc.
- Generación del código final: El compilador traduce el código fuente y el AST (o alguna representación interna) a código de máquina o código intermedio que la computadora pueda ejecutar.

c) La semántica aparece en la etapa de análisis semántico (semántica estática) y es una de las partes más importantes ya que es la previa a la generación de código intermedio. Encuentra los últimos errores que pueden haber arrastrado el análisis léxico y sintáctico. Se enfoca en encontrar el significado de las sentencias.

Ejercicio 3: Con respecto al punto anterior ¿es lo mismo compilar un programa que interpretarlo? Justifique su respuesta mostrando las diferencias básicas, ventajas y desventajas de cada uno.

Diferencias	Intérprete	Compilador
Definición	Programa que ejecuta el código fuente línea por línea sin generar un archivo independiente.	Programa que traduce todo el código fuente a lenguaje de máquina antes de ejecutarlo, creando un archivo ejecutable.
Por cómo se ejecuta	- Se utiliza el intérprete en la ejecución. Ejecuta el programa línea por línea. - Por donde pase dependerá de la acción del usuario, de la entrada de datos y/o de alguna decisión del programa. - El programa fuente será público.	- Se utiliza el compilador antes de la ejecución. - Produce un programa ejecutable equivalente en lenguaje objeto - El programa fuente no será publico
Por el orden de ejecución	Sigue el orden lógico de ejecución (no necesariamente recorre todo el código)	Sigue el orden físico de las sentencias (recorre todo)
Por el tiempo consumido de ejecución	- Por cada sentencia que pasa realiza el proceso de decodificación (lee, analiza y ejecuta) para determinar las operaciones y sus operandos. Es repetitivo. - Si la sentencia está en un proceso iterativo, la tarea de decodificación se realiza tantas veces como sea requerido. La velocidad del proceso se ve afectada por esto mismo.	- Pasa por todas las sentencias. - No repite lazos. - Traduce todo de una sola vez. - Genera código objeto ya compilado. - La velocidad de compilación dependerá del tamaño del código.
Por la eficiencia posterior	- Más lento en ejecución. Se repite el proceso cada vez que se ejecuta el mismo programa o pasa por las mismas instrucciones. - Para ser ejecutado en otra máquina se necesita tener si o si el intérprete instalado y el programa fuente será público.	- Más rápido ejecutar desde el punto de vista del hardware porque ya está en un lenguaje de más bajo nivel. - Detectó más errores al pasar por todas las sentencias. - Siempre está listo para ser ejecutado. - Capaz se tardó más en compilar por las verificaciones previas. El programa fuente no será público.
Por el espacio ocupado	- No pasa por todas las sentencias entonces ocupa menos espacio de memoria. - Cada sentencia se deja en la forma original y las instrucciones interpretadas necesarias para ejecutarlas se almacenan en los subprogramas del interprete en memoria. - Tablas de símbolos, variables y otros se generan cuando se usan en forma dinámica.	- Si pasa por todas las sentencias. - Una sentencia puede ocupar decenas o centenas de sentencias de máquina al pasar a código objeto. - Cosas cómo tablas de símbolos, variables, etc. se generan siempre se usen o no. -El compilador en general ocupa más espacio.
Por la detección de errores	- Las sentencias del código fuente pueden ser relacionadas directamente con la sentencia en ejecución entonces se puede ubicar donde se produjo el error. - Es más fácil detectarlos por donde pasa la ejecución. - Es más fácil corregirlos.	- Se pierde la referencia entre el código fuente y el código objeto. - Es casi imposible ubicar el error, pobres en significado para el programador. - Se deben usar otras técnicas (Semántica Dinámica).
Ventajas	Facilidad de desarrollo y depuración, flexibilidad para realizar cambios en el código sin necesidad de recompilación.	Mayor eficiencia en tiempo de ejecución, mayor capacidad de optimización, independencia del código fuente una vez compilado.
Desventajas	Menor eficiencia en tiempo de ejecución, dependencia continua del código fuente durante la ejecución, falta de optimizaciones globales.	Requiere un paso adicional de compilación antes de la ejecución, lo que puede aumentar el tiempo de desarrollo inicial y el tamaño del archivo ejecutable.

Ejercicio 4: Explique claramente la diferencia entre un error sintáctico y uno semántico. Ejemplifique cada caso.

1) Sintáctico: Ocurre cuando el código no sigue las reglas gramaticales del lenguaje de programación. Son detectados por el analizador sintáctico (Parser) del lenguaje durante la compilación o interpretación, ejemplo en PHP:

```
<?PHP public function CPLP() {
        echo("me aburro")   //Error: Falta ";" después del echo
    } ?>
```

Semántico: Cuando el código está bien a nivel sintáctico pero no produce los resultados esperados debido a una interpretación incorrecta del significado del código, ejemplo en Symfony:

```
<?PHP
    int $test = "Hola";
?>
```

La principal diferencia es que uno rompe en gramática y el otro en lógica.

Ejercicio 5: Sean los siguientes ejemplos de programas. Analice y diga qué tipo de error se produce (Semántico o Sintáctico) y en qué momento se detectan dichos errores (Compilación o Ejecución).
Aclaración: Los valores de la ayuda pueden ser mayores.

No es "var"

• Sintáctico
• Semántico

a) Pascal

```
Program P
var 5: integer;
var a:char;
Begin
    for i:=5 to 10 do begin
        write(a);
        a=a+1;
    end;
End.
```

i

no delimitado

no inicializado

Char + integer

Lo es := , no =

Ayuda: Sintáctico 2, Semántico 3

b) Java:

```
public String tabla(int numero, arrayList<Boolean> listado)
  {
    String result = null;
    for(i = 1; i < 11; i--){
      result += numero + "x" + i + "=" + (i*numero) + "\n";
      listado.get(listado.size()-1)=(BOOLEAN) numero>i;
    }
  return true;
  }
```

• Semántico
• Sintáctico
• Lógico

INT

Bucle infinito, i++

No puede correr con NULL

Acá hay como 4 errores, No sé como marcarlo

comparación ? ==

Debe ser String

* La lista puede estar vacía
* No existe este costeo

Ayuda:
Sintácticos 4, Semánticos 3, Lógico 1

d) Python

```
#!/usr/bin/python
print( "\nDEFINICION DE NUMEROS PRIMOS")
r = 1
while (r = True):
    N = input("\nDame el numero a analizar: ")
    i = 3
    fact = 0
    if (N mod 2 == 0) and (N != 2):
        print "\nEl numero %d NO es primo\n" % N
    else:
        while i <= (N^0.5):
            if (N % i) == 0:
                mensaje="\nEl numero ingresado NO es primo\n" % N
                msg = mensaje[4:6]
                print msg
                fact = 1
            i+=2
        if fact == 0:
            print "\nEl numero %d SI es primo\n" % N

    r = input("Consultar otro número? SI (1) o NO (0)--->> ")
```

• Semántico
• Sintáctico

==

Falta cost

%, no Hay MOD

N**0,5, ^ es XOR

Mismo error, falta el costeo

Lo %d

Ayuda: Sintácticos 2, Semánticos 3

9) Ver resumen de clases teóricas 1 a 3 en este mismo repo :)